



Master 1 (M1)

***Département d'informatique
l'UFR-SEA/
UJKZ***

Bases de Données Avancées

***Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ***

Janvier 2023

<http://kisitokab.xyz>



- **Prérequis**
 - Principes des systèmes de base de données
- **Volume horaire**
 - 30 heures
 - 3 Crédits
- **Méthode d'évaluation**
 - 1 Devoir
 - 1 Projet



Justification du Cours/ module



Résultats de l'apprentissage



Contenu-sujets du Cours

- Introduction à la conception des BdD
- Introduction à la réalisation des BdD
- Introduction à l'administration des BdD
- Les BdD réparties
- Les BdD Objets
- Les extensions XML



Master 1 (M1)
Département d'informatique
l'UFR-SEA /
UJKZ

Bases de Données Avancées

CHAPITRE 6

Bases de Données Orientées Objets

Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ

Janvier 2023

<http://kisitokab.xyz>

Objectifs :

- Les systèmes de BdD OO
- Les 13 règles
- La Norme ODMG
- L'Objet-Relationnel
- Mise en œuvre avec SQL-3
- Mise en œuvre sous ORACLE



Généralités :

Modèles de données et SGBD orientés objet

- **Définitions**
- **MDOO**: Un modèle de données qui capture la sémantique des objets supportée en programmation objet.
- **BDOO**: Une collection persistante et partageable d'objets définis par un MDOO.
- **SGBDOO**: Le gestionnaire d'une BDOO



Généralités :

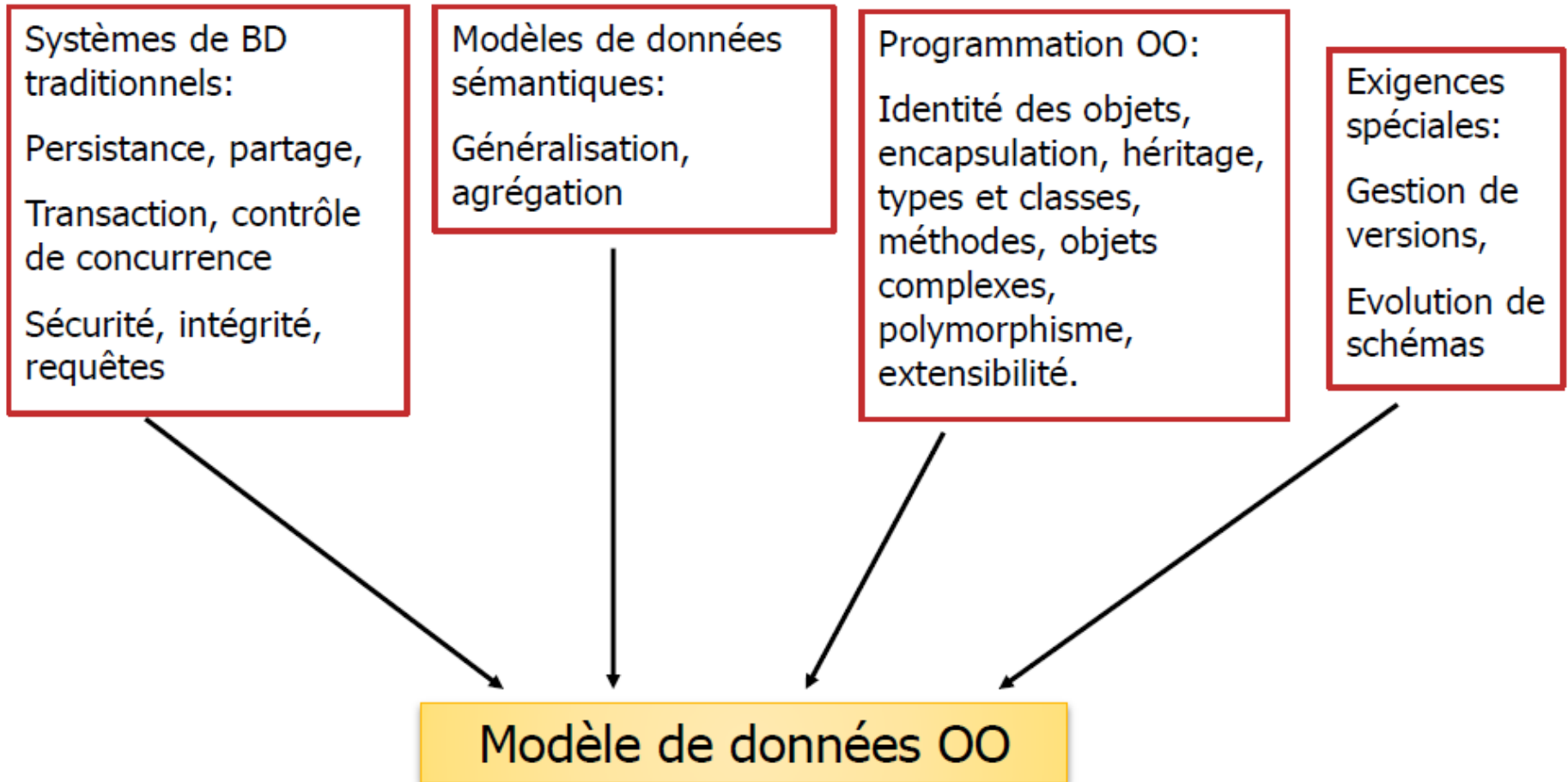
Modèles de données et SGBD orientés objet

- **Fonctionnalités d'un SGBDOO les plus générales**
- A partir de «orientation objet = types de données abstraits + identité d'objet + héritage»
- On aura besoin des fonctionnalités suivantes :
 - ◦Fonctionnalités d'un SGBD classique plus,
 - ◦Il supporte la notion d'identité des objets,
 - ◦Il assure l'encapsulation,
 - ◦Il gère les objets complexes
- *Même si l'héritage est très utile, il n'est pas essentiel à la définition d'un SGBDOO*



Généralités :

Origine du modèle OO



*Par Dr Kaboré Kiswendsida
Kisito.*



Manifeste du système de BD OO

13 règles [Atkinson 1989]

1. Il gère des objets complexes: objets de base + constructeurs:

- ◦ Ensemble minimal de constructeurs:
 - Ensemble (SET)
 - L'occurrence (TUPLE)
 - La liste (LIST ou ARRAY)

2. Il gère l'identité d'objet:

- ◦ Tout objet doit posséder une identité unique, indépendante des valeurs d'un attribut.



Manifeste du système de BD OO

13 règles [Atkinson 1989]

1. Il gère des objets complexes: objets de base + constructeurs:
2. Il gère l'identité d'objet:
3. Il permet l'encapsulation :
 - °Accès aux méthodes qu'à travers l'interface de l'objet.
 - °Données et implémentation des méthodes sont masqués dans l'objet.
4. Il gère les types ou les classes :
 - °Le schéma de BD comporte un ensemble de classes ou de types.



Manifeste du système de BD OO

13 règles [Atkinson 1989]

1. Il gère des objets complexes: objets de base + constructeurs:
2. Il gère l'identité d'objet:
3. Il permet l'encapsulation :
4. Il gère les types ou les classes :
5. Les types ou les classes doivent pouvoir hériter de leurs ancêtres:
 - °Un sous-type ou une sous-classe doit hériter des attributs et des méthodes, respectivement, de son super type ou de sa superclasse.
6. Il doit assurer les liens dynamiques :
 - °Les méthodes s'appliquent à des objets de types différents. A chaque type un code différent, donc la liaison s'effectue à l'exécution: liaison dynamique



Manifeste du système de BD OO

13 règles [Atkinson 1989]

1. Il gère des objets complexes: objets de base + constructeurs:
2. Il gère l'identité d'objet:
3. Il permet l'encapsulation :
4. Il gère les types ou les classes :
5. Les types ou les classes doivent pouvoir hériter de leurs ancêtres:
6. Il doit assurer les liens dynamiques :
7. **Le LMD est complet au niveau calcul :**
 - °Le LMD du SGBDOO doit être un langage de programmation à vocation générale et non pas seulement un langage de requêtes.
8. **L'ensemble des types de données est extensible:**
 - °L'utilisateur doit pouvoir construire de nouveaux types à partir des types prédéfinis par le système.



Manifeste du système de BD OO

+ règles des SGBD traditionnels

1. Il gère des objets complexes: objets de base + constructeurs:
2. Il gère l'identité d'objet:
3. Il permet l'encapsulation :
4. Il gère les types ou les classes :
5. Les types ou les classes doivent pouvoir hériter de leurs ancêtres:
6. Il doit assurer les liens dynamiques :
7. Le LMD est complet au niveau calcul :
8. L'ensemble des types de données est extensible:
- 9. Le système permet la persistance des données :**
 - °Comme dans un SGBD traditionnel, les données doivent continuer d'exister donc persister après la fin de l'application qui les a créées.
- 10. Le SGBD doit être capable de gérer de très vastes bases de données:**
 - °Les données sont stockées en mémoire secondaire, moyennant l'utilisation de méthodes d'accès et de stockage sophistiquées. Le SGBDOO doit disposer de mécanismes semblables.
- 11. Le SGBD accepte des utilisateurs simultanés et concurrents :**
 - °Existence de mécanismes de contrôle de concurrence semblables à ceux des SGBD classiques.



Manifeste du système de BD OO

+ règles des SGBD traditionnels

1. Il gère des objets complexes: objets de base + constructeurs:
2. Il gère l'identité d'objet:
3. Il permet l'encapsulation :
4. Il gère les types ou les classes :
5. Les types ou les classes doivent pouvoir hériter de leurs ancêtres:
6. Il doit assurer les liens dynamiques :
7. Le LMD est complet au niveau calcul :
8. L'ensemble des types de données est extensible:
9. **Le système permet la persistance des données :**
10. **Le SGBD doit être capable de gérer de très vastes bases de données:**
11. **Le SGBD accepte des utilisateurs simultanés et concurrents :**
12. **Le SGBD assure la reprise en cas de défaillances matérielles et/ou logicielles:**
 - Existence de mécanismes de récupération semblables à ceux des SGBD traditionnels.
13. **Le SGBD fournit un moyen simple pour interroger les données :**
 - Existence d'un utilitaire de requêtes de haut niveau (déclaratif), efficace, indépendant de l'application ; (langage ou navigateur graphique)



ODMG :Object Data Management Group

- **ODMG** :Object Data Management Group
- Groupe de normalisation des SGBD OO
- Norme finale publiée en 2001
- A regroupé de nombreux vendeurs de SGBD OO
 - Poet
 - Ardent
 - Objectivity
 - Versant
 - GemStone
- ...
- et des constructeurs, des utilisateurs, des chercheurs
- ...
- www.odmg.org



ODMG : LA NORME ODMG

- LA NORME ODMG :
- Propose un modèle objet pour les objets persistants qui s'appuie sur le modèle objet de l'OMG, et qui définit deux langages:
 - LDO (ODL)
 - LIO(OQL)
 - Interface dans les langages Java, smalltalk et C++.
- Version1 1993, Version3.0 2001
- Un objet est décrit selon quatre caractéristiques:
 - Identificateur : interne au SGBD
 - Nom (optionnel): Utiliser par les applications
 - Durée de vie: transitoire, persistant
 - Structure: un objet est soit atomique soit une collection d'objets



APPORTS DU MODÈLE OBJET

- L'identité d'objet (OID): permet de référence n'importe quel objet dans une base de données orientée objet.
- Encapsulation
- Héritage
- Support des objets complexes, exemples:
 - Tables imbriquées: Une colonne d'une table qui contient une autre table
 - Les données multimédias
 - Les données géographiques (géométriques ou cartographiques)

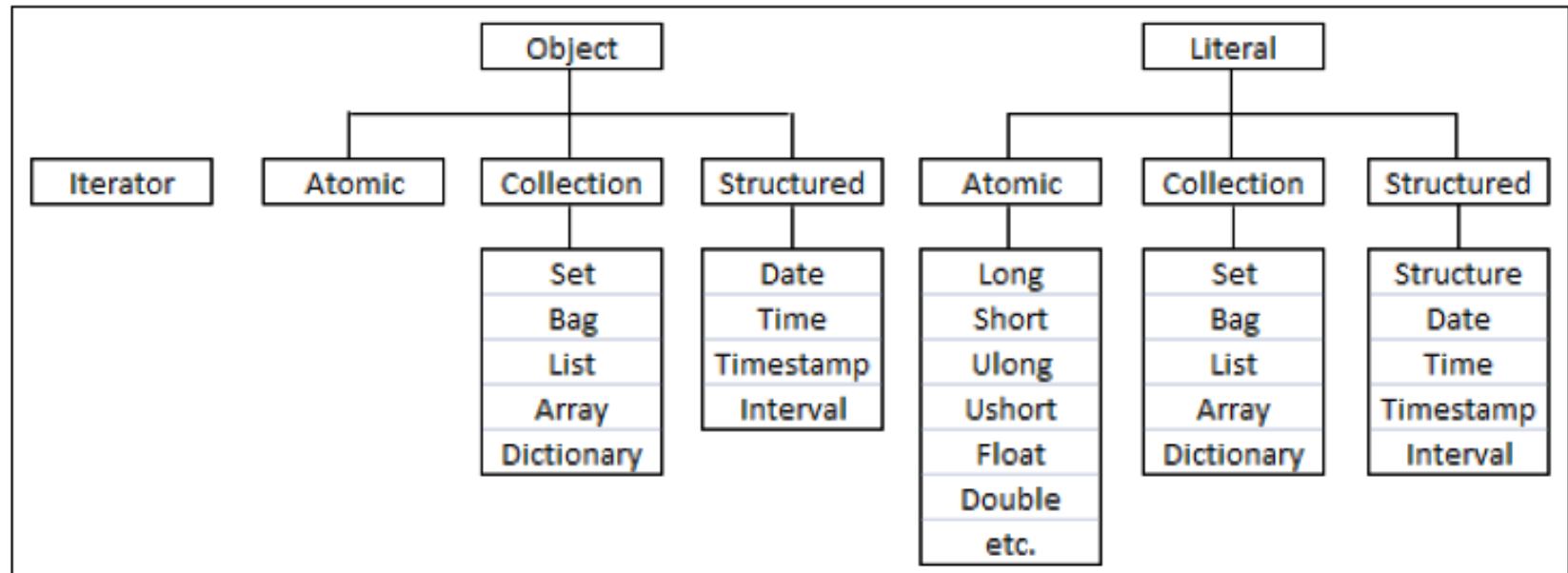


APPORTS DU MODÈLE OBJET

- On note néanmoins qu'il :
 - Manque de bases théoriques dans la systématisation de la conception d'un modèle objet.
 - La navigation entre objets est définie une fois pour toutes.

APPORTS DU MODÈLE OBJET

- Le schéma ci-dessous indique les interfaces de base du modèle ODMG :





DIFFÉRENCE ENTRE SGBOO ET SGBDR

SGBDR	SGBDOO
Table	Classe
Ligne (enregistrement)	instance
Jeu d'enregistrement (ResultSet)	Collection
Colonne (champ)	attribut
Clé primaire	OID
Procédure stockée	Méthode
Relation entre tables	Pointeurs entre objets
Jointure	Navigation

*Par Dr Kaboré Kiswendsida
Kisito.*



Exercices sur ODL et OQL

– LDO (ODL)

– LIO(OQL)



Objectif – 3

Objet-relationalnel

*Par Dr Kaboré Kiswendsida
Kisito.*



Objet-relationnel

- **Limites du modèle relationnel**
- Il y a deux limites principales au modèle relationnel :
 - l'approche orientée objet n'est pas prévue : les concepts d'objet, d'héritage, d'encapsulation, lui sont étrangers.
 - Le modèle relationnel ne manipule que des données classiques. Il n'existe pas de structures de données complexes comme des données multimédias ou des tables par exemple.



Objet-relationnel

- **Deux types de solutions :ont été envisagées**
- 1ère possibilité :
 - abandonner le modèle relationnel et
 - construire un nouveau modèle "objet" de bases de données.
- 2ème possibilité :
 - étendre le modèle relationnel vers un modèle objet-relationnel.
 - Il s'agit de rendre le modèle relationnel compatible avec les "règles" objet.



Objet-relationnel

- De fait, il y a trois extensions essentielles par rapport au modèle relationnel standard :
 - l'incorporation de structures de données complexes : **les types abstraits de données** (TAD en français, ADT en anglais).
 - l'introduction **des objets**, notamment via leur identifiant (appelé **oid : object identifier**)
 - la modélisation des associations



Objet-relationnel

- Les SGBD commerciaux, basés sur le modèle Objet-relationnel sont en général issus du modèle Relationnel. Citons, parmi les plus connus :
- Oracle 8i, 9i
- Sybase
- DB2
- UNISQL
- UNISYS
-



Objet-relationalnel

- **Un système de gestion de base de données (SGBD)** est un ensemble de logiciels qui manipule le contenu des bases de données.
- Il effectue, sur demande, des opérations élémentaires telles que la recherche, l'ajout ou la modification du contenu.
-



Objet-relationnel

- **Selon l'architecture Ansi/Sparc**, la manière dont les informations sont organisées dans un logiciel qui utilise le SGBD peut différer de la manière dont le SGBD présente les informations par l'intermédiaire de l'interface de programmation.
- Cette organisation peut différer de celle qui est utilisée par le SGBD pour stocker les informations sur un support informatique.



Objet-relationnel

- Un SGBD est dit objet (**Système de gestion de base de données Objet / SGBDO**) lorsque les informations sont présentées sous forme d'objets.
- **Un SGBD relationnel** présente les informations sous formes de matrices appelées relations, et
- un **SGBD objet-relationnel** peut présenter les informations alternativement sous formes **de relations ou d'objets**.



Objet-relationnel

- **Les bases de données à objet** sont mises en œuvre avec un SGBD objet ou objet-relationnel, qui est directement exploité par un programme écrit en langage de programmation orientée objet.



Objet-relationnel

- **Les bases de données à objet** sont mises en œuvre avec un SGBD objet ou objet-relationnel, qui est directement exploité par un programme écrit en langage de programmation orientée objet.



Objet-relationnel

SQL3

- Les SGBD qui suivent les prescriptions de SQL3 sont appelés SGBD "Objet-Relationnel".
- SQL3 veut incorporer l'objet tout en gardant les acquis du modèle relationnel (et de SQL2).
- SQL3 permet (nouveau !) la création de types par les utilisateurs. Il en existe 2 catégories : le type DISTINCT, et le
- TAD (Type Abstrait de Données, en anglais ADT) de nature complexe.



Objet-relationnel

SQL3

- CREATE DISTINCT TYPE
- CREATE TYPE
- CREATE ROW TYPE
- CREATE TABLE OF....
-
- *Cf documentation : fichier BdD-Relationnelle-Objet-*

Objet-relational:

Exemple d'objets



Objet-relationnel:

Exemple d'objets

Les types abstraits de données

exemple :

- création du type T_adresse :

```
CREATE TYPE T_adresse ( num NUMBER, voie CHAR(30), ville(30), cp (CHAR(10));
```

- création du type T_personne :

```
CREATE TYPE T_personne (nom CHAR(30), pnom CHAR(30), adr T_adresse);
```

- création d'une table ANNUAIRE :

```
CREATE TABLE ANNUAIRE OF T_personne
```

nom	prenom	adr			
		num	voie	ville	cp

Sur cet exemple, on voit que l'on a incorporé dans une table un objet complexe adr (ce qui est contraire à la 1ère forme normale)

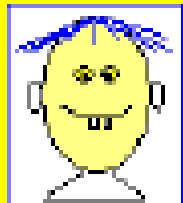
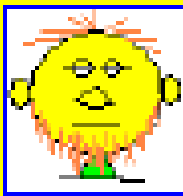
Objet-relational:

Exemple d'objets

Les types abstraits de données

exemple : création d'une table avec des images stockées dans la base de données

CREATE TABLE TROMBINOSCOPE (personne T_personne, photo BLOB);

nom	prenom	adr				photo
		num	voie	ville	cp	
DUPONT	Jacques	3	rue des Jonquilles	Bordeaux	33000	
TERIEUR	Alex	54	avenue des Tomates	Montpellier	34000	
TERIEUR	Alain	78	Cours de la Libération	Nice	06000	



Objet-relationnel:

Exemple d'objets

Les types abstraits de données

exemple :

```
CREATE TYPE T_telephone (tel CHAR(20));  
CREATE TYPE T_personne(nom CHAR(40), pnom CHAR(20), listel SET(T_telephone));  
CREATE TABLE PERSONNE OF T_personne;
```

SET désigne ici une collection d'objets de même type.

nom	<u>prenom</u>	{listel}
		tel
<u>TERIEUR</u>	Alex	03 22 56 78 90
		02 65 43 21 76
		06 07 87 65 43
<u>TERIEUR</u>	Alain	05 98 54 32 76
		05 67 43 56 23
		06 09 65 87 33



Objet-relational:

Exemple d'objets

Les objets

exemple :

```
CREATE TYPE T_adresse (num NUMBER, voie CHAR(30), ville CHAR(30), cp  
CHAR(10));  
CREATE TYPE T_personne(nom CHAR(30), pnom CHAR(30), adr REF T_adresse);  
CREATE TABLE ADRESSE OF T_adresse;  
CREATE TABLE PERSONNE OF T_personne;
```

nom	<u>prenom</u>	@adresse
DUPONT	Jacques	
<u>TERIEUR</u>	<u>Alex</u>	
<u>TERIEUR</u>	Alain	

TABLE PERSONNE

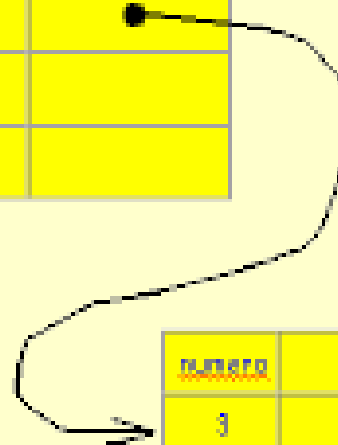


TABLE ADRESSE			
<u>numero</u>	voie	ville	<u>code_postal</u>
3	rue des Jonquilles	Bordeaux	33000
54	avenue des Tomates	Montpellier	34000
78	Cours de la Libération	Nice	06000



Objet-relational:

Exemple d'objets

Les objets

En reprenant l'exemple précédent relatif à la notion de collection, on peut imaginer qu'une collection ne comporte pas un nombre fixé d'éléments et on peut aussi bien en ajouter qu'en supprimer.

exemple : dans le TAD précédent, on peut définir les méthodes `ajouter_tel( )` et `supprimer_tel( )` :

```
CREATE TYPE T_personne(  
    nom CHAR(30),  
    pnom CHAR(50),  
    listel SET(T_telephone),  
    METHOD ajouter_tel(nouveau_tel T_telephone),  
    METHOD supprimer_tel(tel T_telephone)  
);
```

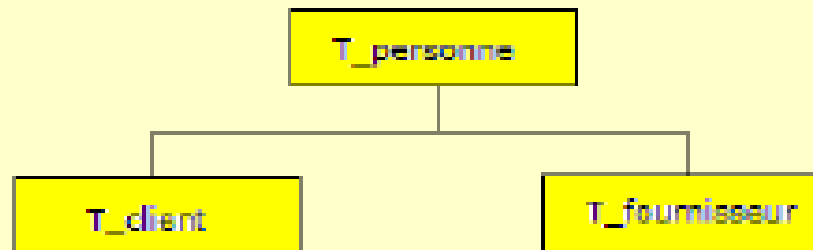
Cet exemple montre que l'**encapsulation** des objets est bien mise en oeuvre. On a donc une analogie assez forte entre type abstrait de données et le concept de classe. Cette analogie concerne aussi l'**héritage** :

exemple : supposons que l'on ait défini le type `T_personne` et que l'on ait deux autres types, `T_client` et `T_fournisseur`, chacun de ces deux types héritant des attributs et des méthodes de `T_personne`.



Objet-relational: Exemple d'objets

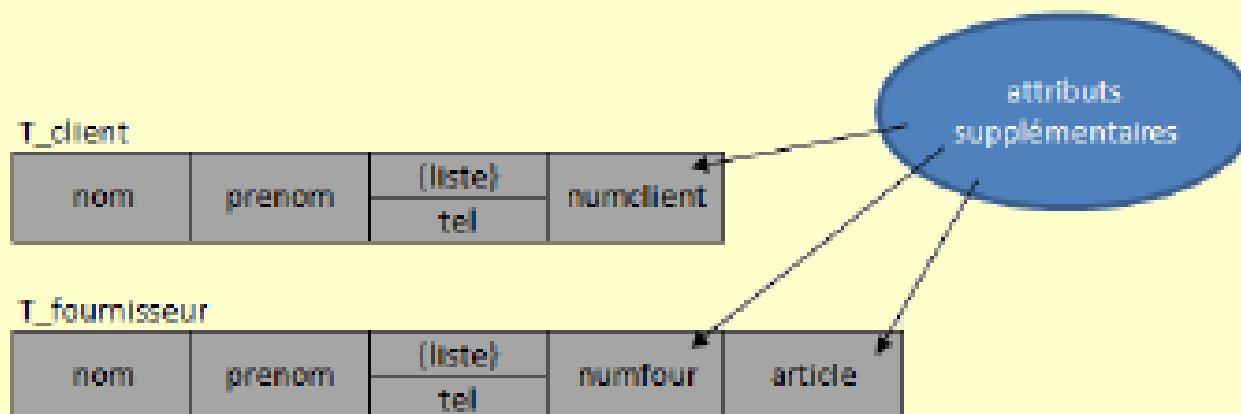
Les objets



On définira les deux types T_client et T_fournisseur comme suit :

```
CREATE TYPE T_client UNDER T_personne(numclient NUMBER);  
CREATE TYPE T_fournisseur UNDER T_personne(numfour NUMBER, article CHAR(80));
```

Les nouveaux types héritent des attributs et méthodes de T personne mais possèdent des attributs spécifiques supplémentaires.



Exemples d'utilisations dans **SQL3**



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les types distinct

Ce type permet une personnalisation des données. Ainsi, un prix d'achat et un prix de vente peuvent s'exprimer par le type prédéfini DOUBLE PRECISION. Mais si l'on veut caractériser ces deux entités (et les "distinguer") , on peut "redéfinir" DOUBLE PRECISION en créant deux "sous-types" :

```
CREATE DISTINCT TYPE T_pachat AS DOUBLE PRECISION;  
CREATE DISTINCT TYPE T_pvente AS DOUBLE PRECISION;  
CREATE TABLE Produit1 (  
    numP VARCHAR(10),  
    pa T_pachat  
    pv T_pvente  
);
```



Objet-relationnel:

Exemple d'utilisations dans SQL3

Le TAD littéral

- Les TAD littéraux portent sur les colonnes des tables relationnelles. On peut regrouper par exemple deux colonnes (dont les champs correspondent à des types différents) en définissant un type pour ce regroupement.
- **exemple** : En reprenant l'exemple précédent, où la table Produit1 possède 3 champs (3 colonnes), on peut regrouper les deux champs pa et pv en définissant un nouveau type :

```
CREATE TYPE T_deuxPrix (  
    achat T_pachat,  
    vente T_pvente );  
CREATE TABLE Produit2 (  
    numP VARCHAR(10),  
    prix T_deuxPrix );  
SELECT p.numP, p.prix.achat FROM Produit2 p WHERE p.prix.vente > 100;
```



Objet-relationnel:

Exemple d'utilisations dans SQL3

Le TAD Objet

- Contrairement aux TAD précédents, les TAD objet concernent les lignes des tables, c'est à dire les tuples. Le mot clé correspondant est ROW.
- **Exemple**
 - **CREATE TYPE T_adresse (num VARCHAR(6), voie VARCHAR(40), ville VARCHAR(30), codepostal INT);**
 - **CREATE ROW TYPE T_personne (numSS INT, nom VARCHAR(30), prenom VARCHAR(30), adresse T_adresse);**
 - **CREATE TABLE Personne OF T_personne(PRIMARY KEY numSS);**
 - **SELECT p.nom, p.prenom, p.adresse FROM Personne p WHERE p.adresse.codepostal=80170;**
- Cette sorte de TAD est appelée "objet" dans la mesure où les lignes d'une table peuvent être considérées comme des objets et peuvent (voir plus loin) être référencées comme une entité indépendante, la référence jouant, en quelque sorte le rôle d'un identifiant d'objet mais n'est toutefois pas identique à un OID.



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les TABLES

- Les tables restent centrales dans SQL3. Tout ce qui concerne les tables en SQL2 est valable en SQL3 (c'est la moindre des choses !).
- On peut toujours créer des tables relationnelles classiques ou des tables relationnelles en NF2 ou encore des tables d'objets.
- *tables relationnelles classiques*
- *exemple :*
- CREATE TABLE Personne (nom VARCHAR(30), prenom VARCHAR(30), telephone NUMBER) ;



Objet-relationalnel:

Exemple d'utilisations dans SQL3

Les TABLES

- *tables relationnelles en NF2*
- Elles peuvent être créées en utilisant des constructeurs de type comme ROW() ou LIST()
- ***exemple :***
 - CREATE TYPE T_adresse(num VARCHAR(5), voie VARCHAR(30), ville VARCHAR(30), codepostal INT);
 - CREATE TABLE Personne (numero INT, adresse T_adresse, identité ROW(nom VARCHAR(30), prenom VARCHAR(30)), postes LIST(ROW(bureau INT, tel VARCHAR(10))) PRIMARY KEY numero);



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les TABLES

- tables d'objets
- Bien que le mot objet n'existe pas dans la syntaxe SQL3, il est implicitement présent dans le modèle lorsque l'on utilise le mot clé OF.
- **exemple :**
 - CREATE ROW TYPE T_personne (nom VARCHAR(30), prenom VARCHAR(30), telephone NUMBER) ;
 - CREATE TABLE Personne OF T_personne (PRIMARY KEY nom) ;
- Dans ce cas, toute instance de la table possède un OID connu du système.



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les constructeurs de type

- Comme leur nom l'indiquent, ils servent à construire des types donnés. Nous en avons vu un exemple précédemment avec ROW() et LIST(). Il existe trois catégories de constructeurs de types :
 - le constructeur ROW qui permet de regrouper un nombre fixe de champs de types différents : le type obtenu est donc un tuple.
 - les constructeurs de collection : SET, LIST, MULTiset qui permettent de regrouper plusieurs éléments de même type
 - le constructeur REF qui permet de définir des référence (pointeurs) vers des objets.



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les constructeurs de type

- *constructeur ROW*
- Il est assez naturel d'emploi et est équivalent à la déclaration ROW TYPE sauf que ROW() s'utilise à l'intérieur de la création d'une table :
- **exemple :**
 - CREATE ROW TYPE T_adresse (num VARCHAR(5), voie VARCHAR(30), ville VARCHAR(30), codepostal INT);
 - CREATE ROW TYPE T_identite(nom VARCHAR(30), prenom VARCHAR(30)); CREATE TYPE T_personne (numero INT, identite T_identite, adresse T_adress);
- est équivalent à
 - CREATE TYPE T_personne (Numero INT, Identite ROW(nom VARCHAR(30), prenom VARCHAR(30)), adresse ROW(num VARCHAR(5), voie VARCHAR(30), ville VARCHAR(30), codepostal INT));
- Le constructeur peut s'utiliser lors d'une instanciation.



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les constructeurs de type

- *constructeurs de collections*
- Une collection contient des éléments de même type (contrairement au ROW). SQL3 définit 3 constructeurs de collection :

collection	doublons	ordre	nb d'éléments
SET	NON	non ordonné	variable
MULTISET	OUI	non ordonné	variable
LIST	OUI	ordonné	variable



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les constructeurs de type

- Constructeur de référence
- Rappelons qu'un élément est considéré comme un objet dès qu'il est placé dans une table avec OF. Un objet possède un oid (identifiant) unique qui est généré automatiquement par le système. Rappelons aussi qu'il est immuable, non réutilisable après disparition de l'objet correspondant.
- Dans SQL3, l'oid d'un objet de type T est référencé par REF(T). Contrairement au modèle O-O, une référence à un objet peut être affichée dans le modèle O-R. Une référence peut être déclarée dans un type ou dans une table.
- **exemple** : supposons deux types définis a priori, un type personne et un type T_equipe (on suppose qu'une personne du type T_personne correspond à une seule équipe du type T_equipe. On désire relier ces deux types par une référence :

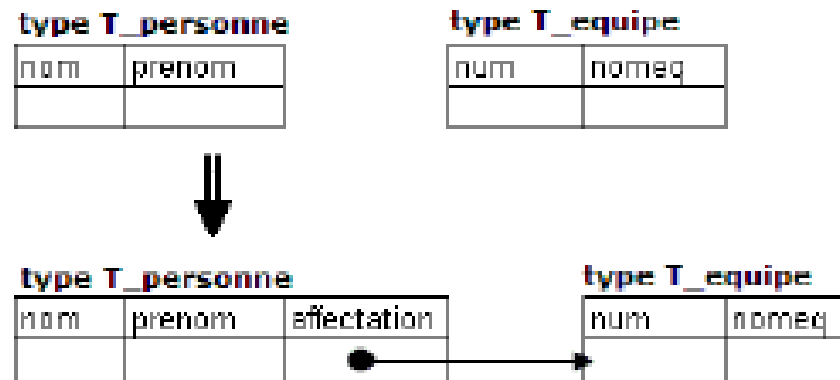


Objet-relationnel:

Exemple d'utilisations dans SQL3

Les constructeurs de type

- Constructeur de référence
 - CREATE OR REPLACE BODY TYPE T_personne (nom VARCHAR(30), prenom VARCHAR(30), affectation REF(T_equipe));
- On peut aussi utiliser le constructeur REF() directement dans la création d'une table.
 - CREATE TABLE binome (personne1 REF(T_personne), personne2 REF(T_personne));





Objet-relat

Exemple d'utilisati

Les constructeurs

type T_personne

nom	prenom

type T_equipe

num	nomeq



type T_personne

nom	prenom	affectation

type T_equipe

num	nomeq

- Constructeur de référence
- Considérons maintenant les deux requêtes suivantes :
 - SELECT p.affectation FROM Personne p WHERE p.nom = 'Dupont';
 - SELECT p.affectation->nomeq FROM Personne p WHERE p.nom = "Dupont";
- La première requête fournira seulement l'adresse de l'équipe correspondant à la personne 'Dupont' ce qui n'est pas en général très utile.
- La seconde requête fournira le nom de l'équipe à laquelle est affecté la personne 'Dupont'. Elle est équivalente à :
 - SELECT e.nomeq FROM Personne p, Equipe e WHERE p.nom = 'Dupont' AND REF(e) = p.affectation;



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les Méthodes

- Nous avons signalé que SQL3 est basé sur un typage fort, c'est à dire que le type inclut les attributs mais aussi les méthodes. Comment SQL3 définit-il les méthodes ?
- Il faut distinguer la déclaration de la définition. La définition s'effectue à la création du type selon la syntaxe évidente :
- CREATE TYPE T_nomtype (attributs, déclaration ou définition des méthodes);
- La définition des méthodes peut se faire lors de la création du type (CREATE) comme la syntaxe précédente le montre ou lorsque l'on modifie le type (REPLACE). La définition d'une méthode nécessite un langage de programmation qui peut être interne(c'est le cas de PL/SQL dans Oracle) ou externe (C ou C++ pu Java).



Objet-relationnel:

Exemple d'utilisations dans SQL3

Les Méthodes

- **exemple** : le type T_personne contient la méthode T_personne() qui ajoute une personne (c'est un constructeur de type !).
- CREATE TYPE T_personne (nom VARCHAR(30), prenom VARCHAR(30),
FUNCTION T_personne(:nom VARCHAR(30), :prenom VARCHAR(30))
RETURNS T_personne; p T_personne; /* déclaration locale */
BEGIN
 :p := T_personne(); /* constructeur */
 p.nom := :nom;
 p.prenom := :prenom;
 RETURN :p;
END);



Objet-relational:

Exemple d'utilisations dans SQL3

Les Méthodes

- La méthode T_personne() est ici définie complètement dans le type. On pourrait seulement la déclarer :

```
CREATE TYPE T_personne (  
  nom VARCHAR(30),  
  prenom VARCHAR(30),  
  FUNCTION T_personne ( :nom VARCHAR(30), :prenom VARCHAR(30)  
  RETURNS T_personne); );
```

- Evidemment il faudra définir, dans ce dernier cas la fonction T_personne() ailleurs, c'est à dire hors du type T_personne.
- SQL3 fournit aussi un certain nombre de fonctions utilitaires comme des fonctions de comparaison et des fonctions de conversion que nous n'explicitons pas ici.



Objet-relationnel:

Exemple d'utilisations dans SQL3

Héritage

- On peut considérer deux types d'héritages : l'héritage de type et l'héritage de table. L'héritage de type permet de définir des sous-types avec le mot clé UNDER.
- **exemple :**
 - CREATE ROW TYPE T_personne (
 nom VARCHAR(30),
 prenom VARCHAR(30)
);
 - CREATE ROW TYPE T_professeur **UNDER** T_personne (
 categorie VARCHAR(10),
 discipline VARCHAR(20)
);
 - CREATE ROW TYPE T_etudiant **UNDER** T_personne (
 diplôme VARCHAR(10),
 numero INTEGER
);
- On notera qu'il suffit d'ajouter les propriétés nouvelles. L'héritage de table s'effectue également avec le mot clé UNDER



Objet-relationnel:

Exemple d'utilisations dans SQL3

Héritage

- ***exemple :***
- CREATE TABLE Personne (
 - nom VARCHAR(30),
 - prenom VARCHAR(30));
-
- CREATE TABLE Professeur UNDER Personne (
 - categorie VARCHAR(10),
 - discipline VARCHAR(20));
-
- CREATE TABLE Etudiant UNDER Personne (
 - diplôme VARCHAR(10),
 - numero INTEGER);

Exemple d'utilisations dans **ORACLE**



Exemple d'utilisations dans ORACLE

- Par rapport aux 13 règles définissant un SGBD Objet, le produit Oracle se positionne de la manière suivante (on se base ici sur Oracle8i) :
 1. objets composites : les types abstraits de données (TAD) définis par l'utilisateur permettent de définir des objets composites.
 2. identités d'objet : le concept d'identifiant existe
 3. encapsulation : oui, mais...
 4. classes, types, méthodes : TAD et tables permettent le concept
 5. héritage : hum....
 6. surcharge et liaison dynamique : oui pour la surcharge
 7. extensibilité : oui
 8. langage de programmation OO : PL/SQL (limité) en interne et C++ et Java en externe.
 9. persistance des données : tout objet stocké dans une base Oracle est persistant.
 10. grande quantité de données : la version 8 peut stocker théoriquement 512x1015 octets.
 11. partage des données et gestion de la concurrence : Oracle propose une architecture NCA (Network Computing Architecture) permettant d'accueillir jusqu'à 15 000 utilisateurs.
 12. fiabilité des données : déclaration de contraintes dans les tables ; déclenchement par triggers.
 13. langage de requête : SQL3 partiellement.



Exemple d'utilisations dans ORACLE

- Oracle fourni les moyens suivants :
 - le type OBJET
 - les types du genre " COLLECTION " : VARRAY et NESTED TABLE

```
CREATE TYPE <nom du type> AS OBJECT (  
    attribut 1 DATATYPE,  
    attribut 2 DATATYPE,  
    -----  
    attribut n DATATYPE,  
    [définition de méthode]  
);
```

```
CREATE TYPE <nom1 du type> AS VARRAY(nmax) OF <nom2 de type>
```

```
CREATE TYPE <nom1 de type> AS TABLE OF <nom2 de type>
```




Objectif – 4

L'ASPECT MULTIMEDIA

*Par Dr Kaboré Kiswendsida
Kisito.*



L'ASPECT MULTIMEDIA

- ***Une donnée multimedia :***
- C'est une donnée qui appartient à plusieurs formats d'information :
 - L'image fixe (dont l'écriture manuscrite)
 - L'image animée
 - Le son
 - La vidéo
 - Le texte alphanumérique (oubli important de Wikipedia).



L'ASPECT MULTIMEDIA

- ***Importance d'une donnée multimédia :***
 - **Exemple d'utilisation : Recherche d'un braqueur de banque par la police.**



L'ASPECT MULTIMEDIA

- ***Quelques problèmes :***
 - le volume de données numérisées est généralement important, ce qui nécessite des techniques de compression/décompression.
 - Les traitements sont plus complexes et nécessitent des matériels et logiciels de plus en plus performants
 - la transmission de données volumineuses nécessite des débits binaires importants sur les réseaux.



L'ASPECT MULTIMEDIA

- *Le multimédia dans l'objet-relationnel*
 - SQL3 propose une solution en définissant des types de grande taille.

●



L'ASPECT MULTIMEDIA

- *Le multimédia dans l'objet-relationnel*
- Les types SQL3 pour le multimedia
- SQL3 a défini des types de données "classiques" :
 - **CHAR(n)** : chaîne de caractères de longueur n
 - **CHAR VARYING(n)** : chaîne de caractères de longueur maximale n [Oracle nomme ce type **VARCHAR2(n)**]
 - **INTEGER** : nombre entier [Oracle nomme ce type **NUMBER**]
 - **NUMERIC(p,q)** ; nombre décimal de p chiffres avec q chiffres décimaux [Oracle nomme ce type **NUMBER(p,q)**]
 - **FLOAT**: nombre flottant [pas d'équivalent chez Oracle]
 - **DATE** : date
 - **TIME** : heure
 - **TIMESTAMP** : date et heure [pas d'équivalent chez Oracle]



L'ASPECT MULTIMEDIA

- *Le multimédia dans l'objet-relationnel*
- **Les types SQL3 pour le multimédia**
- Il a aussi introduit des types de données se référant à des objets multimédias. Ces types sont repris dans Oracle :
 - **BLOB : objet codé en binaire (Binary Large Object)**
 - **CLOB : objet codé en caractères (Character Large Object)**
 - **NCLOB : objet codé en caractères "nationaux" (National Character Large Object)**
 - **BFILE : fichier binaire externe (Binary File)**

L'ASPECT MULTIMEDIA

numero	nom	prenom	photo
1	DUPONT	Jacques	locator1
2	TERIEUR	Alex	locator2
3	TERIEUR	Alain	locator3

fichiers images



```
CREATE TABLE personne  
(  
    numero NUMBER PRIMARY KEY,  
    nom VARCHAR2(20),  
    prenom VARCHAR2(20),  
    photo BLOB  
);
```




L'ASPECT MULTIMEDIA

- ***Structures de données***
- **Les types SQL3 pour le multimedia**
 - Des structures de données multi-dimensionnelles peuvent aussi être utilisées.
 - Exemple : **Les arbres k-dimensionnels (k-d Trees)**
- ***Cf documentations: fichier***



L'ASPECT MULTIMEDIA

- ***Manipulation des données***
- **Des algorithmes de similarité vont être proposés et utilisés pour la recherche des informations.**
- ***Cf documentations: fichier BdD-Relationnelle-MM***



Références

1. ODMG : <http://www.odmg.org/>
2. ODMG : <http://www.objs.com/x3h7/odmg.htm>
3. Universita Autonoma de Madrid ; Object Oriented Programming Concepts :
<http://www.ii.uam.es/~castells/docencia/poo/>
4. Bases de données et SGBD Objet-Relationnel, Gérard-Michel Cochard
5. HEC Suisse : Le langage de requête d'ODMG ; OQL : http://www.hec.unil.ch/gcampono/Teaching/BDA/Slides/BDA09_OQL.pdf#search=%22OQL%22
6. Cisco ; The OQL Language : <http://www.cisco.com/en/US/docs/wireless/cw4mw/>
7. OQL tutorial : <http://www.cse.buffalo.edu/~mpetropo/notes/O2/OQLTutorial.htm>
8. OQL introduction : <http://www.leibek.com/cours/Informatique/SGBD/> POLY%20CH09%20-%20OQL.pdf
9. Modèle objet-relationnel
10. Gamache - Modèle étape : du relationnel vers le modèle objet-relationnel, via le navigationnel - Chapitre 3 <http://www.ift.ulaval.ca/~agamache/IFT19023/H2005/>
11. Gamache - Modèle Objet-Relationnel - Module 4 <http://www.ift.ulaval.ca/%7Eagamache/IFT19023/H2006/ecrans/pageOR.html>
12. Gardarin : l'objet-relationnel <http://www.prism.uvsq.fr/~gardarin/objsql3/sld001.htm>
13. SQL3 Object Model <http://www.objs.com/x3h7/sql3.htm>